STM32 Microcontroller Architecture & Peripherals

1. Overview of STM32F103C8T6 ("Blue Pill")

The STM32F103C8T6 is a high-performance 32-bit microcontroller manufactured by STMicroelectronics, based on the **ARM Cortex-M3** core. It is widely used in industrial, medical, and embedded consumer electronics due to its low cost, rich peripheral set, and deterministic interrupt handling.

---

2. Technical Specifications

Parameter	Value / Details
CPU Core	ARM 32-bit Cortex-M3 (RISC architecture)
Max Clock Frequency	72 MHz
Flash Memory	64 KB (Often 128 KB on C8 variants)
SRAM	20 KB
Operating Voltage	2.0 V to 3.6 V (Nominal: 3.3 V)
5V Tolerant Pins	Yes (many GPIO pins, including **`PB6`**, **`PB7`**, **`PB12`**, **`PB14`**, **`PB15`**)
Timers	3x 16-bit General Purpose, 1x 16-bit Advanced PWM, 2x Watchdogs, SysTick
Communication	2x I2C, 3x USART, 2x SPI, 1x CAN 2.0B, 1x USB 2.0 Full Speed
ADCs	2x 12-bit ADC (16 channels, 1 µs conversion time)
Package	LQFP48 (48 pins)

---

3. Clock Tree Configuration in this Project

In this project (**`SystemClock_Config`**):

1. **Clock Source**: An external 8.000 MHz crystal (HSE - High Speed External) is used.

2. **PLL Multiplier**: The 8 MHz HSE is fed into the Phase-Locked Loop (PLL) with a **`x9`** multiplier:

$$f_{\text{SYSCLK}} = 8\,\text{MHz} \times 9 = 72\,\text{MHz}$$

3. **AHB Prescaler**: **`/1`** $\rightarrow$ $HCLK = 72\,\text{MHz}$ (feeds CPU core, memory, and DMA).

4. **APB1 Prescaler**: **`/2`** $\rightarrow$ $PCLK1 = 36\,\text{MHz}$ (feeds I2C1, Timers 2-4). APB1 max limit is 36 MHz.

5. **APB2 Prescaler**: **`/1`** $\rightarrow$ $PCLK2 = 72\,\text{MHz}$ (feeds GPIO ports A, B, C and Timer 1).

6. **Flash Latency**: 2 wait states (required when $SYSCLK > 48\,\text{MHz}$).

---

4. Peripherals Used in this Project

4.1 I2C1 (Inter-Integrated Circuit)

- **Pins**: `PB6` (SCL) and `PB7` (SDA).
- **Speed Mode**: Standard Mode (100 kHz).
- **Operating Mode**: Master Transmitter / Receiver using HAL blocking functions (`HAL_I2C_Mem_Write`, `HAL_I2C_Mem_Read`).
- **Pin Configuration**: Open-Drain Alternate Function (`GPIO_MODE_AF_OD`) with external/internal pull-up resistors.

4.2 GPIO (General Purpose Input/Output)

- **Button Inputs (`PB12`, `PB14`, `PB15`)**:
- Configured as `GPIO_MODE_INPUT` with internal pull-ups (`GPIO_PULLUP`).
- When unpressed, reading is `GPIO_PIN_SET` (Logic HIGH: 3.3V).
- When pressed, connected directly to ground $\rightarrow$ `GPIO_PIN_RESET` (Logic LOW: 0V).
- **LEDs & Buzzer (`PB1`, `PB10`, `PB0`)**:
- Configured as Push-Pull outputs (`GPIO_MODE_OUTPUT_PP`) with low slew rate (`GPIO_SPEED_FREQ_LOW`).
- Logic HIGH (3.3V) turns on the indicator.

4.3 SysTick Timer

- 24-bit system tick downcounter running at $72\,\text{MHz}$, producing an interrupt every $1\,\text{ms}$.
- Drives `HAL_GetTick()`, used extensively in this project for non-blocking timing (debouncing, blinking, OLED refresh, RTC poll).

---

5. Potential Evaluator Questions & Answers

- **Q: Why choose ARM Cortex-M3 over 8-bit microcontrollers like ATmega328P (Arduino Uno)?**
- *A: Cortex-M3 runs at 72 MHz (4.5x faster), has 32-bit hardware multiply/divide, much larger SRAM (20 KB vs 2 KB), hardware nested vectored interrupt controller (NVIC), and lower power consumption per MIP.*
- **Q: Why is Flash Latency set to 2 wait states?**
- *A: The internal Flash memory cannot deliver instructions as fast as 72 MHz without read errors. The memory interface requires wait states: 0 WS for $0-24\text{ MHz}$, 1 WS for $24-48\text{ MHz}$, and 2 WS for $48-72\text{ MHz}$.*
- **Q: Why configure I2C pins as Alternate Function Open-Drain (`AF_OD`) instead of Push-Pull?**
- *A: In I2C, multiple devices share the line. If a device used push-pull and drove HIGH while another pulled LOW, a short circuit would occur. Open-drain allows any device to pull LOW, while pull-up resistors pull the line HIGH when released (wired-AND logic).*
- **Q: What is the difference between polling and interrupt-based button handling?**
- *A: In this design, buttons are polled every 10 ms with software edge detection and timestamp debounce (`now - lastPress > 200ms`). This avoids interrupt bounce storms and simplifies the finite state machine without blocking the super-loop.*